Alfredo Arnaiz

Feb. 22, 2026

Foundations of Python Programming

Assignment 5

ARArnaiz/IntroToProg-Python-Mod05

Assignment 5: Advanced Collections and Error Handling

Introduction

This document details the process of preparing the required fifth assignment for the Foundations of Python Programming (IT FDN 110 GP Wi 26) course. This assignment builds on the learnings of the first four assignments adding the use of data processing using dictionaries and JSON files.

As in the previous assignments, this one also requires the preparation of a short program following specific criteria regarding acceptance, testing and submission. To accomplish this task, it was required to

1. read the module notes and watch the listed videos, and
2. read and watch additional materials related to this assignment.

Additionally, his time the submission included:

3. posting the relevant files to GitHub
4. posting the link to the GitHub site into the Discussion board, and
5. perform a peer review.

Script/Program Preparation

Given the instructions and acceptance criteria, this is the process I followed:

- Review and renaming of the starter file *Assignment05-Starter.py*.
- Updated the header information.
- Added the import statement for the JSON module
- Reviewed and updated the required data constants and variables.

- Set up the reading/loading of the JSON file.
- Review the conditional logic underlying the menu system and the corresponding input prompts, actions and outputs. This time some structured error handling code was included, and the output data was written to a JSON file.

There are number of aspects I wish to comment on:

- **Commented out the *csv_data* variable** – Since this version outputs a list of dictionary records into a JSON file, there is no need to use this variable.
- **Multi-line f-string for console output for menu choices (1-3)** – In order to avoid exceeding the recommended line length (i.e. 89 characters), I reformatted part of the output messages in the print statements with two f-strings, see figures (1-3):

```
print(
    f"You have registered {student_first_name} {student_last_name} "
    f"for {course_name}."
)
```

Figure 1 - Print statement for menu choice (1)

```
print("-" * 50)
for student in students:
    print(
        f"Student {student['FirstName']} {student['LastName']} "
        f"is enrolled in {student['CourseName']}"
    )
print("-" * 50)
```

Figure 2 - Print statements for menu choice (2)

```
print("The following data was saved to file!")
for student in students:
    print(
        f"Student {student['FirstName']} {student['LastName']} "
        f"is enrolled in {student['CourseName']}"
    )
```

Figure 3 - Print statements for menu choice (3)

- **Menu choice (2) print statements** - It was unclear to me if there were supposed to be one or two print statements (see Mod05-Assignment document, third and fourth bullets of the Input / Output subsection in the Acceptance Criteria section). I decided to keep only one, the first one, not be repetitive. Additionally, the second one was about CSV data that we were not using in this program anymore.
- **Structured Error Handling**
    - **File read** – Added three potential exception statements:

- *FileNotFoundError* deals with the case in which the required file doesn't exist.
- *json.decoder.JSONDecodeError* deals with the case of a malformed JSON input file.
- The generic catch all: Non-specific error.

- **First/last name** – I experimented with using one or two error handling try blocks.
  - One try block – It seemed economical, but the error on *first name* skips the *last name* line of code and continued with the *course name*, which I found confusing. See figure – 4:

```
---------- Course Registration Program ----------
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
-------------------------------------------------

What would you like to do: 1
Enter the student's first name: 23
First name must be a string of letters only!
-- Technical Error Message --
Inappropriate argument value (of correct type).
First name must be a string of letters only!
Please enter the name of the course:
```

Figure 4 - Capture of the console output for the error handling of a first name issue.

  - Two try blocks – This option seemed more robust since it treated the potential input errors on first and last name separately. See figure – 5.

```
---------- Course Registration Program ----------
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
-------------------------------------------------

What would you like to do: 1
Enter the student's first name: 23
First name must be a string of letters only!
-- Technical Error Message --
Inappropriate argument value (of correct type).
First name must be a string of letters only!
Enter the student's last name: 23
Last name must be a string of letters only!
-- Technical Error Message --
Inappropriate argument value (of correct type).
Last name must be a string of letters only!
Please enter the name of the course:
```

Figure 5 - Capture of the console output for the error handling when first and last name have their own try block.

- **File write** – Added two potential exception statements:
  - *TypeError* deals with invalid JSON format.
  - The generic catch all: Non-specific error.

- **JSON file readability** – I added the option *indent = 4* to the JSON.dump statement to help with readability. This option provides a valid JSON format that is easier to read. See Appendix 2 for a capture of the JSON output file.
- **Error Handling and data validation question** – I observed while testing that the structure error handling alerts the user of potential data entry and other errors, but it doesn't necessarily offer a way to validate the data. See figure - 6. Is there a simple way to guard entering invalid data?

```
---------- Course Registration Program ----------
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
-------------------------------------------------

What would you like to do: 1
Enter the student's first name: 23
First name must be a string of letters only!
-- Technical Error Message --
Inappropriate argument value (of correct type).
First name must be a string of letters only!
Enter the student's last name: 23
Last name must be a string of letters only!
-- Technical Error Message --
Inappropriate argument value (of correct type).
Last name must be a string of letters only!
Please enter the name of the course: 4
You have registered 23 23 for 4.
```

Figure 6 - Capture of the console output after error handling for first and last name; notice that the invalid data is recorded.

Testing

Besides testing as I developed the script, I tested full runs on

- PyCharm and
- PyCharm's terminal (equivalent to command prompt testing).

In both cases, I tested a first pass over the menu option "1" twice, followed by option "2". Next, I saved the data to the file through menu option "3". In addition, I tested all outputs including the JSON file. No unexpected results. See the Appendices for the output of the PyCharm run.

In addition, I tested the structured error handling options by triggering the errors expected to catch. See above for the error handling about the first and last name input strings. For

the read checks, I tested without the input file in the expected folder (see figure – 7) and also with an empty file (see figure – 8):

```
C:\Users\ARA\Documents\Python\PythonCourse\Module05\.venv\Scripts\python.exe C:\Users\ARA\Documents\Python\PythonCourse\Module05\Assignment05.py
File must exist before running this script!

-- Technical Error Message --
[Errno 2] No such file or directory: 'Enrollments.json'
File not found.
<class 'FileNotFoundError'>
```

Figure 7 – Capture of error message for missing file

```
C:\Users\ARA\Documents\Python\PythonCourse\Module05\.venv\Scripts\python.exe C:\Users\ARA\Documents\Python\PythonCourse\Module05\Assignment05.py
Data file is empty or not a valid JSON file!

-- Technical Error Message --
Expecting value: line 1 column 1 (char 0)
Subclass of ValueError with the following additional properties:

msg: The unformatted error message
doc: The JSON document being parsed
pos: The start index of doc where parsing failed
lineno: The line corresponding to pos
colno: The column corresponding to pos


<class 'json.decoder.JSONDecodeError'>
```

Figure 8 – Capture of error message for empty file

For the write/JSON.dump check, I tested after making the JSON file read-only, see figure – 9:

```
C:\Users\ARA\Documents\Python\PythonCourse\Module05\.venv\Scripts\python.exe C:\Users\ARA\Documents\Python\PythonCourse\Module05\Assignment05.py

---------- Course Registration Program ----------
  Select from the following menu:
    1. Register a Student for a Course.
    2. Show current data.
    3. Save data to a file.
    4. Exit the program.
-------------------------------------------------

What would you like to do: 3
-- Technical Error Message --
Built-In Python error info:
[Errno 13] Permission denied: 'Enrollments.json'
Not enough permissions.
<class 'PermissionError'>
The following data was saved to file!
Student Bob Smith is enrolled in Python 100
Student Sue Jones is enrolled in Python 100
```

Figure 9 – Capture of error message with a read-only JSON file at write attempt

Finally, I did the, now standard, style checking often while developing within PyCharm using the “reformat code” command (Ctrl+Alt+L).

Summary

This document details the process of preparing the required fifth assignment for the Foundations of Python Programming (IT FDN 110 GP Wi 26) course. This assignment involved testing a course registration program using both PyCharm and its terminal interface. Menu options were tested systematically, including registering students, displaying current data, and saving information to a JSON file. All outputs, including the JSON file, were closely verified, and no unexpected results were observed. Error handling features were rigorously checked by intentionally triggering known errors, such as missing files and empty files, with corresponding error messages captured and documented (see Figures 7 and 8). Additionally, write operations were tested with a read-only JSON file to confirm proper exception handling (see Figure 9).

Throughout development, code style checks were performed frequently using PyCharm's "reformat code" command to ensure readability and maintainability. The results of the test runs, including program execution, menu navigation, and error handling, are provided in the appendices for reference. Overall, the program performed as expected, with robust error handling and clean code formatting verified in multiple scenarios.

Appendix 1: PyCharm Test Run

```
C:\Users\ARA\Documents\Python\PythonCourse\Module05\.venv\Scripts\python.exe
C:\Users\ARA\Documents\Python\PythonCourse\Module05\Assignment05.py


---------- Course Registration Program ----------

 Select from the following menu:

   1. Register a Student for a Course.

   2. Show current data.

   3. Save data to a file.

   4. Exit the program.

-------------------------------------------------


What would you like to do: 1

Enter the student's first name: Marc

Enter the student's last name: Bonardelli

Please enter the name of the course: Econ-220

You have registered Marc Bonardelli for Econ-220.
```

```
---------- Course Registration Program ----------
 Select from the following menu:
   1. Register a Student for a Course.
   2. Show current data.
   3. Save data to a file.
   4. Exit the program.
--------------------------------------------------

What would you like to do: 1
Enter the student's first name: Adam
Enter the student's last name: Smith
Please enter the name of the course: Econ-550
You have registered Adam Smith for Econ-550.

---------- Course Registration Program ----------
 Select from the following menu:
   1. Register a Student for a Course.
   2. Show current data.
   3. Save data to a file.
   4. Exit the program.
--------------------------------------------------

What would you like to do: 2
--------------------------------------------------
Student Bob Smith is enrolled in Python 100
Student Sue Jones is enrolled in Python 100
Student Marc Bonardelli is enrolled in Econ-220
Student Adam Smith is enrolled in Econ-550
--------------------------------------------------

---------- Course Registration Program ----------
 Select from the following menu:
   1. Register a Student for a Course.
```

2. Show current data.

3. Save data to a file.

4. Exit the program.

What would you like to do: 3

The following data was saved to file!

Student Bob Smith is enrolled in Python 100

Student Sue Jones is enrolled in Python 100

Student Marc Bonardelli is enrolled in Econ-220

Student Adam Smith is enrolled in Econ-550

---------- Course Registration Program ----------

Select from the following menu:

1. Register a Student for a Course.

2. Show current data.

3. Save data to a file.

4. Exit the program.

What would you like to do: 4

Program Ended

Process finished with exit code 0

Appendix 2: Captures of File Content in PyCharm

```
[
{"FirstName": "Bob", "LastName": "Smith", "CourseName":  "Python 100"},
{"FirstName": "Sue", "LastName": "Jones", "CourseName":  "Python 100"}
]
```

Figure 10 - Capture of initial content from the Enrollments.json file

```
[
    {
        "FirstName": "Bob",
        "LastName": "Smith",
        "CourseName": "Python 100"
    },
    {
        "FirstName": "Sue",
        "LastName": "Jones",
        "CourseName": "Python 100"
    },
    {
        "FirstName": "Marc",
        "LastName": "Bonardelli",
        "CourseName": "Econ-220"
    },
    {
        "FirstName": "Adam",
        "LastName": "Smith",
        "CourseName": "Econ-550"
    }
]
```

Figure 11 – Capture of final content of the output Enrollments.json file